

Báo cáo bài tập

Họ và tên : **Đặng Huy Hiếu**

MSSV : 20223771

Lớp : Kỹ thuật Vi xử lý

Mã lớp : 173836

GVHD : **TS. Hàn Huy Dũng**

Đề bài

- Viết chương trình đơn giản: in ra 1 dòng “hello world” bằng 3 ngôn ngữ: Python, C , Assembly.
 - Dịch ra 3 file exe chạy độc lập. So sánh kích thước 3 file.
 - Chạy 3 chương trình trên máy tính của mình. So sánh thời gian chạy.
 - Viết báo cáo toàn bộ quá trình.
-

Báo cáo kết quả đạt được

1. Viết chương trình “Hello world”

Python

Trong Python, để viết ra chương trình “Hello world”, thì đơn giản chỉ cần 1 dòng duy nhất:

```
print("Hello world")
```

Lưu file main.py, và sau đó chạy lệnh:

```
python3 main.py
```

Câu lệnh được chạy trên OS Debian 13, cú pháp trên Windows đơn giản là `python main.py` Lưu ý: cần chuyển về folder python/ trước khi chạy `cd python`

Kết quả:

```
Hello World
```

C/C++

- Với C, chương trình như sau:

```
#include <stdio.h>

int main () {
    printf("Hello world");
}
```

Chương trình lưu trong `helloc.c`

Chạy chương trình:

```
crg helloc.c
```

Kết quả:

```
Hello world%
```

- Với C++, chương trình như sau:

```
#include <iostream>

int main() {
    std::cout << "Hello world" << std::endl;
}
```

Chương trình lưu trong file `hellocpp.cpp`

Chạy chương trình:

```
crg hellocpp.cpp
```

Kết quả:

```
Hello world
```

Assembly

Assembly là 1 ngôn ngữ khá mới đối với bản thân em, nên em đã tham khảo code trên trang web [TutorialsPoint](#)

Chương trình như sau:

```
section .text:
    global _start

_start:
    mov edx, len
    mov ecx, msg
    mov ebx, 1
    mov eax, 4
```

```
int 0x80

mov eax, 1
mov ebx, 0
int 0x80

section .data
msg db 'Hello world', 0xa
len equ $ - msg
```

Chương trình được lưu trong file `hello.asm`

Các điều kiện để chương trình được biên dịch, liên kết và cuối cùng có thể “chạy” file thực thi:

- Đảm bảo câu lệnh sử dụng ở folder chứa `hello.asm`
- Máy tính đã cài đặt `nasm` (*chi tiết xem ở phụ lục*)
- Để “tập hợp” chương trình, sử dụng câu lệnh `nasm -f elf hello.asm`
- Sau bước trên, folder xuất hiện file `hello.o` (*nếu không thì có lỗi*)
- Để liên kết object file và tạo file thực thi, chạy lệnh `ld -m elf_i386 -s -o hello hello.o`
- Sau khi bước trên thành công, file `hello` sẽ được tạo, chạy bằng cách `./hello`

Một vài lưu ý:

- Hệ điều hành sử dụng là Debian 13, vậy nên file `hello.asm` sẽ rất khác với file này trên windows, và đồng thời các câu lệnh cũng rất khác, tham khảo phụ lục.
- Ta có thể thay thế tên file thực thi output bằng bất cứ cái tên nào, chẳng hạn `... -o hellobian13 ...`

Chạy file `hello` bằng câu lệnh `./hello` trên terminal ở folder chứa file, kết quả thu được:

```
Hello world
```

2. Dịch ra 3 file thực thi độc lập, so sánh kích thước và tốc độ

Vì hệ điều hành em sử dụng là Debian 13, nên file thực thi là file elf (executable & linkable format).

Để lấy được kích thước file, em sử dụng câu lệnh: `du -csh $file`, trong đó `-csh` là flag gồm: `-c`: in thêm tổng, `-s`: in tổng của đối tượng chỉ định (không liệt kê đối tượng con), `-h`: hiển thị **“human-readable”**, và `$file` là file hoặc folder mà muốn kiểm tra kích thước.

Để so sánh tốc độ chạy file thực thi, em sử dụng thêm gói `hyperfine` có sẵn trên `apt`: `sudo apt install hyperfine`.

Python

Python là một ngôn ngữ lập trình thường được sử dụng như một ngôn ngữ scripting, tức là file `.py` chỉ cần chứa các dòng lệnh cơ bản và đôi khi là các hàm với chức năng phức tạp hơn, còn việc chạy hay thực thi file là do python runtime thực hiện. Ở trên Debian 13, `python3` được cài thông qua lệnh `sudo apt install python3`, là một interpreter, tức là nó chịu trách nhiệm thực thi các file `.py`. Vì vậy, việc tạo file thực thi cho chương trình python là tương đối khó khăn, thứ nhất là phải có package phù hợp để đóng gói python runtime và tất cả dependencies cần thiết để tạo 1 file thực thi và thứ hai là kích thước của file đó cũng vì vậy mà lớn hơn file thực thi của C/C++ rất nhiều.

Trong báo cáo này, em sử dụng `pyinstaller`. Cách dùng:

```
pyinstaller --one-file main.py
```

Gói `pyinstaller` sẽ thực thi đóng gói python runtime để tạo thành 1 file thực thi `main` duy nhất (ở đường dẫn `./python/dist/main`)

Kích thước file:

```
// Câu lệnh
du -csh python/dist/main
```

```
// Kết quả
7.8M    python/dist/main
7.8M    total
```

Tốc độ chạy file:

```
// Câu lệnh
hyperfine -N -r 100 ./python/dist/main

// Kết quả
Benchmark 1: ./python/dist/main
  Time (mean ± σ):      104.9 ms ±   1.9 ms
    [User: 94.1 ms, System: 10.1 ms]
  Range (min ... max):  103.2 ms ... 113.2 ms
  100 runs
```

C/C++

Với việc sử dụng câu lệnh tùy chỉnh `crg` (xem thêm ở phụ lục) trong `~/.zshrc`, đầu ra của câu lệnh này đã là 1 file thực thi:

```
crg c cpp/hello.c
crg c_cpp/hello.cpp
```

Kích thước file:

```
// Giả sử đang ở folder Bai_1 (folder gốc)
du -a c cpp/

// Kết quả
4      ./hello.c
16     ./hello
```

```
4      ./hellocpp.cpp
20     ./hellocpp
```

Tốc độ file:

```
// helloc
hyperfine -N -r 100 ./c_cpp/helloc

// Kết quả
Benchmark 1: ./Bai 1/c cpp/helloc
  Time (mean ± σ):      298.4 μs ±  54.3 μs
    [User: 253.0 μs, System: 7.3 μs]
  Range (min ... max):  259.5 μs ... 626.1 μs
    100 runs

// hellocpp
hyperfine -N -r 100 ./c_cpp/hellocpp

// Kết quả
Benchmark 1: ./Bai 1/c cpp/hellocpp
  Time (mean ± σ):      770.4 μs ±  85.0 μs
    [User: 664.6 μs, System: 37.8 μs]
  Range (min ... max):  608.0 μs ... 1006.3 μs
    100 runs
```

Assembly

Kích thước file thực thi là kích thước file hello (elf):

```
// Giả sử đang ở folder Bai_1 (folder gốc)
du -a asm/

// Kết quả
4  asm/hello.asm
8  asm/hello
4  asm/hello.o
20 asm
```


Tốc độ file:

```
// Câu lệnh
hyperfine -N -r 100 ./Bai_1/asm/hello

// Kết quả
Benchmark 1: ./Bai 1/asm/hello
  Time (mean ± σ):      309.9 μs ± 105.1 μs
    [User: 188.4 μs, System: 13.0 μs]
  Range (min ... max):  212.8 μs ... 849.4 μs
  100 runs
```

Nhận xét về kết quả đạt được

Đây là bảng so sánh các thông số của các chương trình:

Chương trình	Kích thước	Thời gian (min/max/avg)
Python	7.8 MB	103.2/113.2/104.9 (ms)
C	16 B	259.5/626.1/298.4 (μs)
C++	20 B	608/1006.3/770.4 (μs)
Assembly	8 B	212.8/849.4/309.9 (μs)

Nhìn chung, file thực thi của python có kích thước cũng như thời gian lớn nhất, và giảm dần theo thứ tự là C++, C và Asm.

Có thể thấy, ngôn ngữ lập trình nào càng dễ viết, càng yêu cầu nhiều bộ nhớ và thời gian thực thi.

Phụ lục

Về assembly trong windows và linux

Câu lệnh mà có thể chạy code `asm` trên windows: `nasm hello.asm -f bin -o hello.com` và sau đó chạy DOS emulator, ví dụ như DOSBox: `dosbox hello.com`

Nếu muốn workflow cũng giống trên `linux` (`[nasm] -> .o file -> [linker] -> elf file`) thì có thể chạy `nasm -f win64 -o hello.asm` và sau đó dùng `gcc` làm linker.

crg

`crg` là 1 alias function được định nghĩa trong `~/ .zshrc` của em:

```
# Compile & Run with gcc/g++
crg () {
    if [ -z "$1" ]; then
        echo "[ERROR]: No files found"
        return 1
    fi

    ext="${1##*.}"
    ext=$(echo "$ext" | tr '[:upper:]' '[:lower:]')

    filename="${1%.*}"
    version="${2:-11}"

    if [ "$ext" = "cpp" ]; then
        ggg="g++"
        std="c++"
    elif [ "$ext" = "c" ]; then
        ggg="gcc"
        std="c"
    else
        echo "[ERROR]: False extension"
        return 1
    fi
}
```

```
fi

"$ggg" -std="$std$version" -O3 -Wall "$1" -o
"$filename" && "./$filename"
}
```

Cách dùng: `crg $file $version`

Cài đặt `nasm`

`nasm` là 1 công cụ mã nguồn mở, có sẵn trong kho `apt` của Debian 13, hoặc có thể tìm trên Windows qua kho `scoop`.

Ngoài ra, có thể sử dụng `FASM` hoặc `GAS`.

Tài liệu phiên bản pdf

Báo cáo này được viết dưới định dạng `.md` (markdown) file.

Sau đó em sử dụng công cụ `pandoc` và `weasyprint` để lần lượt convert `report.md` sang `report.md.html` và `report.md.pdf`.

Tài liệu tham khảo

[pyinstaller](#)

[TutorialsPoint](#)

[Hyperfine](#)

[Pandoc](#)

[Weasyprint](#)